

Local-computation optimization — before / after study

Date: 2026-05-31. Methodology: five rounds of paired A/B testing across three Claude Code skills, with the model held constant per cell so the delta isolates the optimization from background model drift.

Companion PDF: [skills-optim-study-2026-05-31.pdf](#) . Raw data: [skills-optim-study-2026-05-31.json](#) (Round 1), [-round2.json](#) , [-rounds-3-4-5.json](#) .

tl;dr

Round	Skills measured	Cells	Aggregate	Headline
1	skills-freshness, skills-quality	6	+2.5%	3 of 6 cells sign-flipped — discovered the cost-trap mechanisms
2	skills-freshness (post-fix)	3	+11%	all 3 flipped cells turned around after fix
3	content-audit (rigor-exempt test)	3	+1%	rigor-exempt label inflated, sat at break-even
4	content-audit (post-fix)	3	+4%	MIXED — haiku +16pp flip, sonnet -6pp regression
5	both new skills (re-validation)	6	+16%	MIXED — strongest aggregate, concentrated in two haiku cells

42 sub-agents total, ~4.48M subagent tokens spent measuring.

The principle holds: "maximize local computation, minimize LLM tokens" is a real optimization with measurable effect. **The fix is model-dependent:** smaller models (Haiku) benefit reliably and substantially; larger models (Opus) show variable response. **N=1 per cell** is sufficient to surface direction but not magnitude.

Why this study exists

The 2026-05-22 [Spacepotatis skills calibration](#) measured a ~22% net savings rate across 13 skills, replacing the editorialial 3× heuristic with real numbers. That study answered "how much do my skills save?" — a portfolio-level question.

This study answers a different question: **for a specific optimization round (PRs #14–#17 on [claude-skills](#)), did the optimization principle actually work?** It's a same-model, same-task A/B with the model held constant within each cell, so the delta reflects only the optimization — not model drift, not natural per-session variance, not the harness's cache state from earlier work. Stronger isolation than the calibration's cross-session aggregation; narrower scope.

Round 1 — original measurement (2 skills × 2 arms × 3 models)

Hypothesis: the two new skills ([skills-freshness](#) , [skills-quality](#)) should net-save tokens vs unguided improvisation.

Cell	BEFORE	AFTER	Saved	%	Verdict
skills-freshness/sonnet	194,751	110,828	+83,923	+43%	strongest save
skills-freshness/opus	134,765	163,301	–28,536	–21%	sign-flip
skills-freshness/haiku	64,192	108,806	–44,614	–70%	largest sign-flip

Cell	BEFORE	AFTER	Saved	%	Verdict
skills-quality/sonnet	97,928	109,465	-11,537	-12%	sign-flip
skills-quality/opus	56,615	53,895	+2,720	+5%	marginal
skills-quality/haiku	81,544	67,623	+13,921	+17%	save
Aggregate	629,795	613,918	+15,877	+2.5%	

3 of 6 cells went negative. The +2.5% headline was honest-but-small. The forensic per-cell pass found three mechanisms, visible in the JSONL transcripts:

- read-granularity regression:** skills-freshness/haiku AFTER read 233,876 chars across 14 SKILL.md files (no `limit` param) vs BEFORE's 116,731 — because the procedure said "read each SKILL.md" literally.
- post-script review depth:** skills-freshness/opus AFTER spent 4 extra Bash calls chasing broken path refs into the source repo, costing +28K tokens.
- parallel-batch staging:** skills-quality/sonnet AFTER staged a "read in parallel" instruction as 6+8 files, paying +27K `cache_creation` tokens.

Each fired because the SKILL.md procedure **removed scoping pressure** the improvised BEFORE arm had.

Round 2 — post-fix validation on skills-freshness

`claude-skills` PR #18 tightened `skills-freshness/SKILL.md` step 3 with the exact guard wording that targets the three mechanisms: `limit=80`, `no deep read`, `max 3 traces total`, `don't spelunk`. Round 2 re-ran the skills-freshness A/B (`skills-quality/SKILL.md` was unchanged, so skipping its cells avoided pure measurement noise).

Cell	R1 %	R2 %	Swing	Note
skills-freshness/sonnet	+43%	+14%	-29pp	still positive, BEFORE got cheaper
skills-freshness/opus	-21%	+1%	+22pp	sign-flip cleared
skills-freshness/haiku	-70%	+20%	+90pp	largest single swing in study

Aggregate skills-freshness: +3% → +11%. Hypothesis confirmed on all three cells. The `limit=80` guard killed haiku's 233K-char full-file read. The "max 3 traces, don't spelunk" cap stopped opus's source-repo hunt. Sonnet's BEFORE happened to be cheaper this run, so the relative save shrunk — but sign and direction unchanged.

Round 3 — rigor-exempt label test on content-audit

The new cost-trap rules flagged a group of "rigor-exempt" skills where we dismissed the findings on the assumption that cost-IS-the-point — five tracked in [issue #20](#), drawn from the broader seven-skill set this study's Round-1 scope skipped (`rigor_exempt_skills_skipped` in the raw JSON). Round 3 tested that assumption on `content-audit` (Spacepotatis project skill, single-phase, no nested sub-agents — the cleanest test case in that skipped set).

Cell	BEFORE	AFTER	Saved	%
content-audit/sonnet	138,012	142,301	-4,289	-3%
content-audit/opus	162,332	149,141	+13,191	+8%
content-audit/haiku	93,281	98,137	-4,856	-5%

Cell	BEFORE	AFTER	Saved	%
Aggregate	393,625	389,579	+4,046	+1%

Finding: content-audit sat at break-even, not deep-negative as the rigor-exempt label assumed. The label was inflated. Wholesale exemption from cost-trap rules would silence a genuine signal.

Round 4 — post-fix re-run of content-audit

Added "at most 5 traces, batch into a single grep with alternation if more, don't spelunk per-id" to step 2 of content-audit's SKILL.md.

Goal stated up front:

- **PASS:** ≥ 2 of 3 cells flip AND aggregate $> +5\%$
- **MIXED:** 1 cell flips AND aggregate moves but stays $< +5\%$
- **FAIL:** 0 cells flip OR aggregate regresses

Cell	R3 %	R4 %	Swing	Flipped?
content-audit/sonnet	-3%	-9%	-6pp	no — regressed
content-audit/opus	+8%	+10%	+2pp	no (already +)
content-audit/haiku	-5%	+11%	+16pp	YES (neg → pos)
Aggregate	+1%	+4%	+3pp	—

Verdict: MIXED. Haiku's +16pp mirrors Round 2's freshness/haiku result (+90pp on a similar cap fix) — smaller models reliably benefit from bounded procedural language. Sonnet regressed -6pp, plausibly N=1 noise from the added wording's procedural complexity. Aggregate moved positive but stayed under the +5% PASS threshold. (Opus was already positive going in, so only sonnet and haiku were candidates to flip — the " ≥ 2 of 3 cells flip" PASS bar effectively required both; one did.)

Two N=1 caveats apply to this table: R3 and R4 used independent BEFORE arms (content-audit's cold-walk baseline drifted +7–13% per model between the two runs), so the per-cell pp swings fold in baseline variance, not just the fix's effect — the opus +2pp in particular sits inside its own ~13% baseline drift. The robust signal is the haiku sign-flip, which is large enough to survive that noise.

Fix shipped anyway as [Spacepotatis PR #280](#).

Round 5 — re-validation of both new skills

The question: are Rounds 1 + 2 actually repeatable, or did we tell a coherent story from N=1 noise?

Goal stated up front:

- **PASS:** skills-freshness ≥ 2 cells positive AND skills-quality direction matches Round 1 AND aggregate > 0
- **MIXED:** one of the two passes
- **FAIL:** both deviate from prior rounds

Cell	R1 %	R2 %	R5 %	Notes
skills-freshness/sonnet	+43%	+14%	+1%	positive each round, magnitude declining
skills-freshness/opus	-21%	+1%	-1%	sign-flip cleared, hovers near break-even

Cell	R1 %	R2 %	R5 %	Notes
skills-freshness/haiku	−70%	+20%	+48%	consistent strong saver post-fix
skills-quality/sonnet	−12%	—	+2%	flipped positive (no R2 data)
skills-quality/opus	+5%	—	−62%	N=1 anomaly — BEFORE was 31K (short-circuited)
skills-quality/haiku	+17%	—	+54%	strongest haiku save in study
Aggregate (6 cells)	—	—	+16%	strongest aggregate of any round

Verdict: MIXED per the strict goal. skills-freshness passed (2 of 3 cells positive). skills-quality didn't directionally match Round 1 (1 of 3 cells match sign). But aggregate is the strongest of any round (+16% across the 6 cells / 12 agents).

The +16% is concentrated, not broad-based. The two haiku cells (skills-freshness +52,289, skills-quality +55,623) sum to ~108K — more than the entire net save of 90,288 — while the other four cells net **−17,624** (freshness/sonnet +1,548, freshness/opus −1,921, quality/sonnet +2,195, quality/opus −19,446). Read "strongest aggregate" as "two strong haiku saves outweighing four flat-to-negative cells," not an across-the-board gain. This is what ratio-of-sums weighting surfaces (see Method → How aggregates are computed).

The opus/skills-quality −62% is an N=1 anomaly: BEFORE cell was 31K tokens (opus short-circuited cold without thoroughly auditing); AFTER (51K) was the more representative number.

Cross-round synthesis

Stable findings across all five rounds

- **Haiku consistently benefits most.** Round 1 +17% on quality, Round 2 +20% on freshness, Round 4 +16pp flip on content-audit, Round 5 +48% on freshness and +54% on quality. The investigation-collapse pattern is the most reliable finding in the study.
- **Aggregate trend is positive and growing — on a composition-matched basis.** The headline aggregates (R1 +2.5%, R2 +11%, R5 +16%) are *not* a like-for-like series: R1 and R5 are 6 cells (skills-freshness + skills-quality), R2 is 3 cells (skills-freshness only — skills-quality was skipped), and Rounds 3–4 measure a different skill entirely (content-audit, +1% / +4%). The comparable skills-freshness-only ratio-of-sums does climb cleanly: **+2.7% (R1) → +10.7% (R2) → +14.9% (R5)**. So the principle pays off harder as fixes land *when you hold the skill set constant*; the raw headline progression also reflects changing composition.
- **Opus is the most variable model:** single-cell swings up to **67pp** between rounds (skills-quality/opus +5% in R1 → −62% in R5). Measurement noise dominates Opus at N=1; do not read individual Opus cells as confident signal.

What the five rounds collectively show

The "maximize local computation, minimize LLM tokens" principle is real and the cost-trap rules in `skills-quality` (claude-skills PR #18) detect genuine waste. The fix's effectiveness is **model-dependent**:

- **Smaller models (Haiku) benefit reliably and substantially.** Haiku is the round's largest-saving cell **by % saved** in three of five rounds (R2 +20%, R4 +11% flip, R5 +54%). (By absolute tokens R4's largest is opus; R2 and R5 hold under both metrics.) In the two rounds where it isn't (R1, R3) it was actually a *negative* cell — so "reliable beneficiary" describes the post-fix pattern, not a clean sweep; pre-fix, bounded guidance can cost haiku more than it saves (R1 freshness/haiku −70%).

- **Larger models (Opus) show variable response.** Sometimes a small save, sometimes an N=1 anomaly. Don't over-interpret single Opus cells.

N=1 per cell is sufficient to surface direction but not magnitude. Headline numbers should always be read with that caveat. Wholesale rigor-exempt labels are inflated; per-finding judgment beats blanket dismissal.

Implications for the cost-trap rules

The three rules added in claude-skills PR #18 (`unlimited_read_in_procedure`, `uncapped_followup`, `batch_invitation`) detect a real class of waste, validated on at least one cell each. But the rule messages slightly overclaim universality. A future iteration could note "fix likely helps haiku-class models substantially; may modestly hurt opus-class models that don't over-execute" so users know what to expect.

The five remaining rigor-exempt skills in [issue #20](#) are still labeled exempt **by assumption only** — each would benefit from a Round-3-style test before the dismissal is treated as durable. Round 3 on content-audit showed the label was inflated for at least one such skill.

Method

A/B design (per cell)

Two fresh sub-agents per cell, same task statement, same target directory, same hour, same model. Differ only in whether the agent received the current SKILL.md.

- **BEFORE** arm: task only, no procedural guidance. Solves cold.
- **AFTER** arm: task plus current SKILL.md. Follows the procedure.

Each agent returned a structured report (skills examined, findings count, tool-use estimate, approach, summary) via JSON schema for consistent comparison.

Token accounting

Per-agent JSONL transcripts under `~/.claude/projects/.../subagents/workflows/*/agent-*.jsonl`. For each assistant message, summed `usage.input_tokens + usage.cache_creation_input_tokens + usage.output_tokens`, deduped by `(sessionId, requestId)` (per agent, so `requestId` alone is equivalent). `cache_read_input_tokens` excluded — those were paid upstream. Same convention as `scripts/build-review-stats.mjs`.

How aggregates are computed

Every round's aggregate % is **ratio-of-sums**: net tokens saved across all cells divided by total BEFORE tokens — volume-weighted, so high-baseline cells count more. It is **not** the mean of the per-cell percentages, and the two diverge sharply at N=1. Round 1's +2.5% ratio-of-sums becomes **-6.3%** as an unweighted mean of its six cell percentages; Round 5's +16% becomes **+7%**. Ratio-of-sums answers "did this batch of runs get cheaper overall," which is the right question for a portfolio-cost lens — but every headline % here should be read as a token-weighted figure, not a typical-cell figure.

Why same-model, same-hour

The skill-registry's aggregated stats fold in transcripts from different sessions, different models, and different weeks. Useful for portfolio-level cadence; dangerous as an isolation of any single optimization's effect. This study pins the model, task, and hour so any delta reflects the SKILL.md change alone.

BEFORE arm definition

For these new skills, the parent commit of the optimizing PR did not contain the SKILL.md at all, so BEFORE = no SKILL.md. The arm represents what an unguided LLM would do given only the task.

Workflow run record

Round	Workflow ID	Agents	Wall-clock	Subagent tokens
1	wf_69f7e105-98f	12	244 s	1,131,817
2	wf_8323de85-910	6	190 s	642,172
3	wf_1f7f067c-791	6	573 s	758,141
4	wf_9c93fac3-e6b	6	610 s	828,450
5	wf_0d56c8c9-5bc	12	212 s	1,122,667
Total		42	~30 min	~4.48M

The **Subagent tokens** column is each run's workflow-harness-reported total, captured from the `wf_*` run record at completion. It uses a different accounting/dedupe scope than the per-cell BEFORE/AFTER sums recomputed from the JSONL transcripts in the round tables, so the two do not tie out exactly (they differ by up to ~10% per round, in both directions). The per-cell token figures are the authoritative measurement behind every save and percentage in this study; this column is a rough cost-of-running-the-study tally only.

Fixes shipped between rounds

- [MikkoNumminen/claude-skills PR #18](#) — cost-trap rules + skills-freshness SKILL.md tightening (between Round 1 and Round 2)
- [MikkoNumminen/claude-skills PR #19](#) — clears 3 cleared-this-round findings on ai-codegen-smell-audit, readme-drift-sync, skill-calibration (between Round 2 and Round 3)
- [MikkoNumminen/claude-skills issue #20](#) — rigor-exempt dismissal tracking
- [MikkoNumminen/Spacepotatis PR #280](#) — content-audit cap (between Round 3 and Round 4)

What this study does NOT claim

- That the +16% aggregate generalizes to the broader portfolio. Three skills were measured; this study isn't the calibration.
- That the "minimize LLM tokens" principle is invalidated. Three of six original cells confirmed it; the conditions matter.
- That maintenance cost of writing the new cost-trap rules and the SKILL.md edits is folded in. It isn't.
- That per-model dollar pricing is included. Raw token totals only.
- That a second N=1 re-run would land within 5% of these numbers. With per-cell variance up to 67pp on Opus, the precision bound is wider than we can pin from one observation.